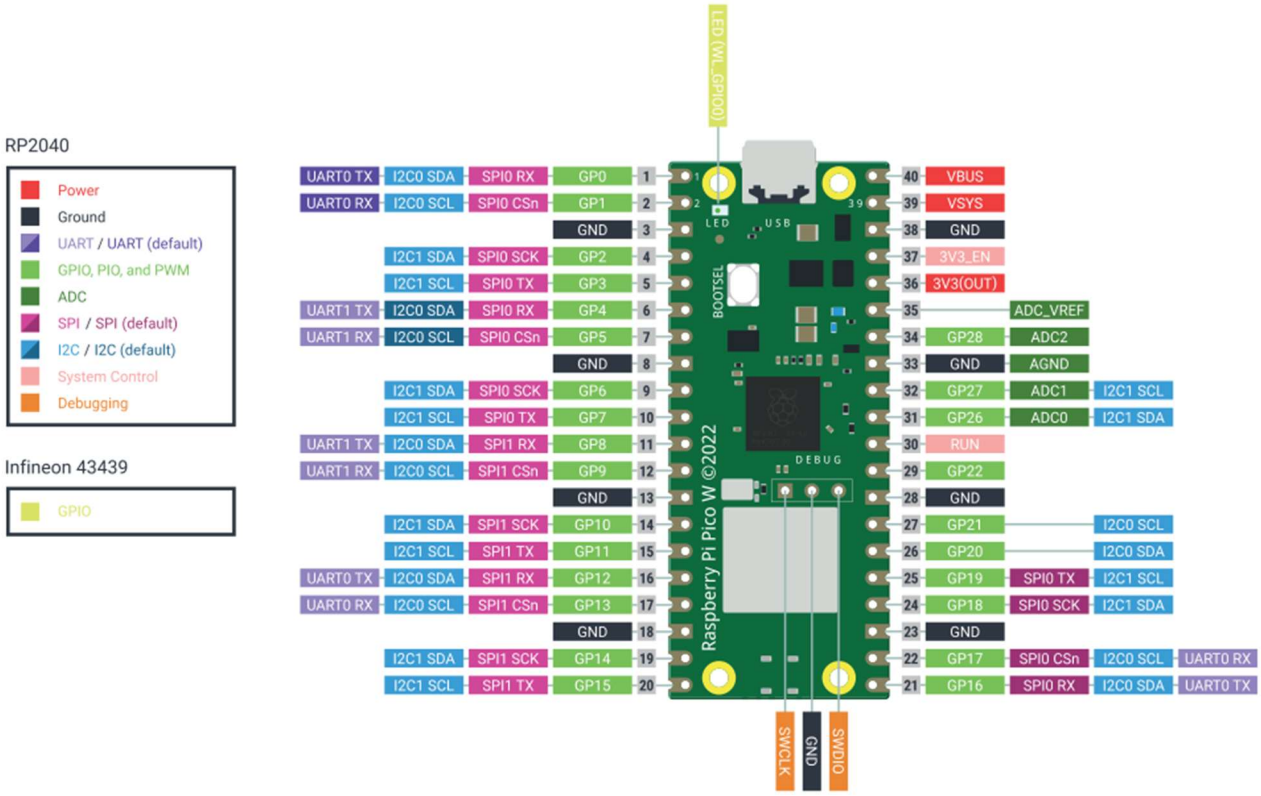
Hands-on Practice Assignments: Week 3



Experiment 10: Interface I2C LCD with Pico W to display some custom characters

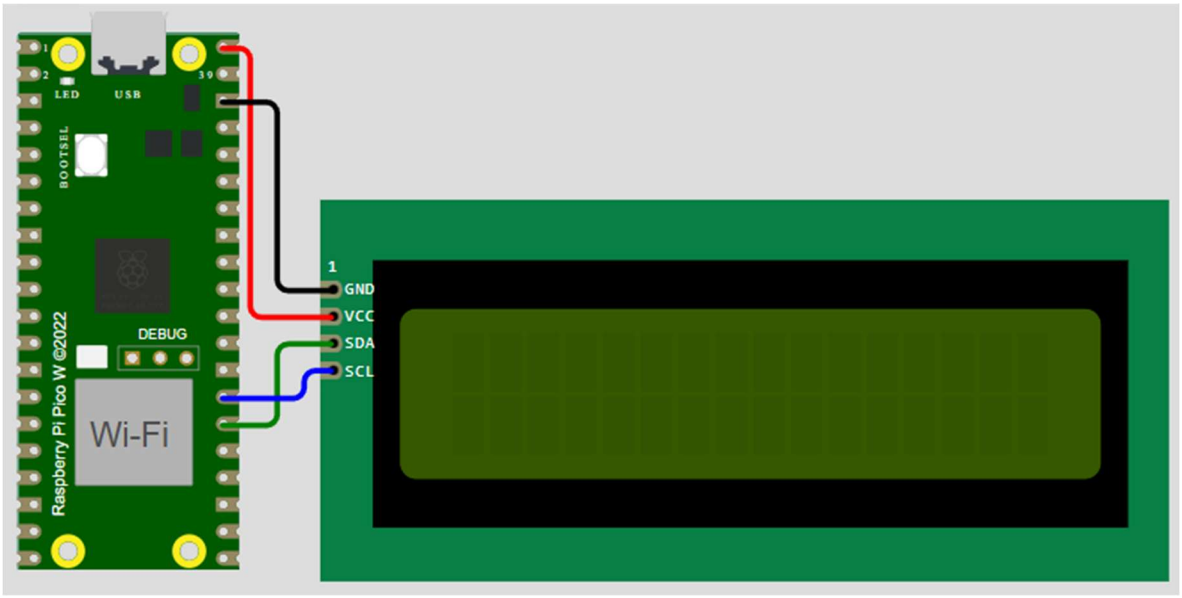


Fig 5: Circuit Diagram for 10

#Code

#main.py

```
import utime
from machine import I2C, Pin
from lcd_api import LcdApi
from pico_i2c_lcd import I2cLcd
I2C_ADDR     = 39
I2C_NUM_ROWS = 2
I2C_NUM_COLS = 16
sda = machine.Pin(20)
scl = machine.Pin(21)

i2c = I2C(0, sda=sda, scl=scl, freq=400000)
lcd = I2cLcd(i2c, I2C_ADDR, I2C_NUM_ROWS, I2C_NUM_COLS)

def greeting():
    lcd.clear()
    lcd.move_to(3,0)
    lcd.putstr("Welcome To")
    lcd.move_to(1,1)
    lcd.putstr("AICTE Idea Lab")
    utime.sleep(2)
    lcd.clear()

def customcharacter():
  #character
  lcd.custom_char(0, bytearray([
  0x0E,
  0x0E,
  0x04,
  0x1F,
  0x04,
  0x0E,
  0x0A,
  0x0A    ]))

    #character2
  lcd.custom_char(1, bytearray([
    0x1F,
  0x15,
  0x1F,
  0x1F,
  0x1F,
  0x0A,
  0x0A,
  0x1B    ]))
```

```
    #smiley
  lcd.custom_char(2, bytearray([
  0x00,
  0x00,
  0x0A,
  0x00,
  0x15,
  0x11,
  0x0E,
  0x00    ]))

  #heart
  lcd.custom_char(3, bytearray([
   0x00,
  0x00,
  0x0A,
  0x15,
  0x11,
  0x0A,
  0x04,
  0x00    ]))

    #note
  lcd.custom_char(4, bytearray([
   0x01,
  0x03,
  0x05,
  0x09,
  0x09,
  0x0B,
  0x1B,
  0x18    ]))
    #celcius
  lcd.custom_char(5, bytearray([
  0x07,
  0x05,
  0x07,
  0x00,
  0x00,
  0x00,
  0x00,
  0x00    ]))

while True:
  greeting()
  customcharacter()
  lcd.move_to(0,0)
```

```
lcd.putstr("Custom Character")
lcd.move_to(0,1)
lcd.putchar(chr(0))
lcd.move_to(4,1)
lcd.putchar(chr(1))
lcd.move_to(8,1)
lcd.putchar(chr(2))
lcd.move_to(12,1)
lcd.putchar(chr(3))
lcd.move_to(15,1)
lcd.putchar(chr(4))
utime.sleep(4)
```

#lcd_api.py

```
import time
class LcdApi:

    # Implements the API for talking with HD44780 compatible character LCDs.
    # This class only knows what commands to send to the LCD, and not how to get
    # them to the LCD.
    #
    # It is expected that a derived class will implement the hal_xxx functions.
    #
    # The following constant names were lifted from the avrlib lcd.h header file,
    # with bit numbers changed to bit masks.

    # HD44780 LCD controller command set
    LCD_CLR             = 0x01  # DB0: clear display
    LCD_HOME            = 0x02  # DB1: return to home position

    LCD_ENTRY_MODE      = 0x04  # DB2: set entry mode
    LCD_ENTRY_INC       = 0x02  # DB1: increment
    LCD_ENTRY_SHIFT     = 0x01  # DB0: shift

    LCD_ON_CTRL         = 0x08  # DB3: turn lcd/cursor on
    LCD_ON_DISPLAY      = 0x04  # DB2: turn display on
    LCD_ON_CURSOR       = 0x02  # DB1: turn cursor on
    LCD_ON_BLINK        = 0x01  # DB0: blinking cursor

    LCD_MOVE            = 0x10  # DB4: move cursor/display
    LCD_MOVE_DISP       = 0x08  # DB3: move display (0-> move cursor)
    LCD_MOVE_RIGHT      = 0x04  # DB2: move right (0-> left)

    LCD_FUNCTION        = 0x20  # DB5: function set
    LCD_FUNCTION_8BIT   = 0x10  # DB4: set 8BIT mode (0->4BIT mode)
    LCD_FUNCTION_2LINES = 0x08  # DB3: two lines (0->one line)
```

```
LCD_FUNCTION_10DOTS = 0x04  # DB2: 5x10 font (0->5x7 font)
LCD_FUNCTION_RESET  = 0x30  # See "Initializing by Instruction" section

LCD_CGRAM           = 0x40  # DB6: set CG RAM address
LCD_DDRAM           = 0x80  # DB7: set DD RAM address

LCD_RS_CMD          = 0
LCD_RS_DATA         = 1

LCD_RW_WRITE        = 0
LCD_RW_READ         = 1

def __init__(self, num_lines, num_columns):
    self.num_lines = num_lines
    if self.num_lines > 4:
        self.num_lines = 4
    self.num_columns = num_columns
    if self.num_columns > 40:
        self.num_columns = 40
    self.cursor_x = 0
    self.cursor_y = 0
    self.implied_newline = False
    self.backlight = True
    self.display_off()
    self.backlight_on()
    self.clear()
    self.hal_write_command(self.LCD_ENTRY_MODE | self.LCD_ENTRY_INC)
    self.hide_cursor()
    self.display_on()

def clear(self):
    # Clears the LCD display and moves the cursor to the top left corner
    self.hal_write_command(self.LCD_CLR)
    self.hal_write_command(self.LCD_HOME)
    self.cursor_x = 0
    self.cursor_y = 0

def show_cursor(self):
    # Causes the cursor to be made visible
    self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY |
                           self.LCD_ON_CURSOR)

def hide_cursor(self):
    # Causes the cursor to be hidden
    self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY)

def blink_cursor_on(self):
    # Turns on the cursor, and makes it blink
```

```
        self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY |
                               self.LCD_ON_CURSOR | self.LCD_ON_BLINK)

    def blink_cursor_off(self):
        # Turns on the cursor, and makes it no blink (i.e. be solid)
        self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY |
                               self.LCD_ON_CURSOR)

    def display_on(self):
        # Turns on (i.e. unblanks) the LCD
        self.hal_write_command(self.LCD_ON_CTRL | self.LCD_ON_DISPLAY)

    def display_off(self):
        # Turns off (i.e. blanks) the LCD
        self.hal_write_command(self.LCD_ON_CTRL)

    def backlight_on(self):
        # Turns the backlight on.

        # This isn't really an LCD command, but some modules have backlight
        # controls, so this allows the hal to pass through the command.
        self.backlight = True
        self.hal_backlight_on()

    def backlight_off(self):
        # Turns the backlight off.

        # This isn't really an LCD command, but some modules have backlight
        # controls, so this allows the hal to pass through the command.
        self.backlight = False
        self.hal_backlight_off()

    def move_to(self, cursor_x, cursor_y):
        # Moves the cursor position to the indicated position. The cursor
        # position is zero based (i.e. cursor_x == 0 indicates first column).
        self.cursor_x = cursor_x
        self.cursor_y = cursor_y
        addr = cursor_x & 0x3f
        if cursor_y & 1:
            addr += 0x40    # Lines 1 & 3 add 0x40
        if cursor_y & 2:    # Lines 2 & 3 add number of columns
            addr += self.num_columns
        self.hal_write_command(self.LCD_DDRAM | addr)

    def putchar(self, char):
        # Writes the indicated character to the LCD at the current cursor
        # position, and advances the cursor by one position.
        if char == '\n':
```

```
            if self.implied_newline:
                # self.implied_newline means we advanced due to a wraparound,
                # so if we get a newline right after that we ignore it.
                pass
            else:
                self.cursor_x = self.num_columns
        else:
            self.hal_write_data(ord(char))
            self.cursor_x += 1
        if self.cursor_x >= self.num_columns:
            self.cursor_x = 0
            self.cursor_y += 1
            self.implied_newline = (char != '\n')
        if self.cursor_y >= self.num_lines:
            self.cursor_y = 0
        self.move_to(self.cursor_x, self.cursor_y)

    def putstr(self, string):
        # Write the indicated string to the LCD at the current cursor
        # position and advances the cursor position appropriately.
        for char in string:
            self.putchar(char)

    def custom_char(self, location, charmap):
        # Write a character to one of the 8 CGRAM locations, available
        # as chr(0) through chr(7).
        location &= 0x7
        self.hal_write_command(self.LCD_CGRAM | (location << 3))
        self.hal_sleep_us(40)
        for i in range(8):
            self.hal_write_data(charmap[i])
            self.hal_sleep_us(40)
        self.move_to(self.cursor_x, self.cursor_y)

    def hal_backlight_on(self):
        # Allows the hal layer to turn the backlight on.
        # If desired, a derived HAL class will implement this function.
        pass

    def hal_backlight_off(self):
        # Allows the hal layer to turn the backlight off.
        # If desired, a derived HAL class will implement this function.
        pass

    def hal_write_command(self, cmd):
        # Write a command to the LCD.
        # It is expected that a derived HAL class will implement this function.
        raise NotImplementedError
```

```
    def hal_write_data(self, data):
        # Write data to the LCD.
        # It is expected that a derived HAL class will implement this function.
        raise NotImplementedError

    def hal_sleep_us(self, usecs):
        # Sleep for some time (given in microseconds)
        time.sleep_us(usecs)
```

 # pico_i2c_lcd.py

```
import utime
import gc

from lcd_api import LcdApi
from machine import I2C

# PCF8574 pin definitions
MASK_RS = 0x01       # P0
MASK_RW = 0x02       # P1
MASK_E  = 0x04       # P2

SHIFT_BACKLIGHT = 3  # P3
SHIFT_DATA      = 4  # P4-P7

class I2cLcd(LcdApi):

    #Implements a HD44780 character LCD connected via PCF8574 on I2C

    def __init__(self, i2c, i2c_addr, num_lines, num_columns):
        self.i2c = i2c
        self.i2c_addr = i2c_addr
        self.i2c.writeto(self.i2c_addr, bytes([0]))
        utime.sleep_ms(20)   # Allow LCD time to powerup
        # Send reset 3 times
        self.hal_write_init_nibble(self.LCD_FUNCTION_RESET)
        utime.sleep_ms(5)    # Need to delay at least 4.1 msec
        self.hal_write_init_nibble(self.LCD_FUNCTION_RESET)
        utime.sleep_ms(1)
        self.hal_write_init_nibble(self.LCD_FUNCTION_RESET)
        utime.sleep_ms(1)
        # Put LCD into 4-bit mode
        self.hal_write_init_nibble(self.LCD_FUNCTION)
        utime.sleep_ms(1)
        LcdApi.__init__(self, num_lines, num_columns)
        cmd = self.LCD_FUNCTION
        if num_lines > 1:
```

```
            cmd |= self.LCD_FUNCTION_2LINES
        self.hal_write_command(cmd)
        gc.collect()

    def hal_write_init_nibble(self, nibble):
        # Writes an initialization nibble to the LCD.
        # This particular function is only used during initialization.
        byte = ((nibble >> 4) & 0x0f) << SHIFT_DATA
        self.i2c.writeto(self.i2c_addr, bytes([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytes([byte]))
        gc.collect()

    def hal_backlight_on(self):
        # Allows the hal layer to turn the backlight on
        self.i2c.writeto(self.i2c_addr, bytes([1 << SHIFT_BACKLIGHT]))
        gc.collect()

    def hal_backlight_off(self):
        #Allows the hal layer to turn the backlight off
        self.i2c.writeto(self.i2c_addr, bytes([0]))
        gc.collect()

    def hal_write_command(self, cmd):
        # Write a command to the LCD. Data is latched on the falling edge of E.
        byte = ((self.backlight << SHIFT_BACKLIGHT) |
                (((cmd >> 4) & 0x0f) << SHIFT_DATA))
        self.i2c.writeto(self.i2c_addr, bytes([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytes([byte]))
        byte = ((self.backlight << SHIFT_BACKLIGHT) |
                ((cmd & 0x0f) << SHIFT_DATA))
        self.i2c.writeto(self.i2c_addr, bytes([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytes([byte]))
        if cmd <= 3:
            # The home and clear commands require a worst case delay of 4.1 msec
            utime.sleep_ms(5)
        gc.collect()

    def hal_write_data(self, data):
        # Write data to the LCD. Data is latched on the falling edge of E.
        byte = (MASK_RS |
                (self.backlight << SHIFT_BACKLIGHT) |
                (((data >> 4) & 0x0f) << SHIFT_DATA))
        self.i2c.writeto(self.i2c_addr, bytes([byte | MASK_E]))
        self.i2c.writeto(self.i2c_addr, bytes([byte]))
        byte = (MASK_RS |
                (self.backlight << SHIFT_BACKLIGHT) |
                ((data & 0x0f) << SHIFT_DATA))
        self.i2c.writeto(self.i2c_addr, bytes([byte | MASK_E]))
```

```
        self.i2c.writeto(self.i2c_addr, bytes([byte]))
        gc.collect()
```

Experiment 11: Interface 0.96inch I2C OLED with Pico W to display temperature from internal temperature sensor

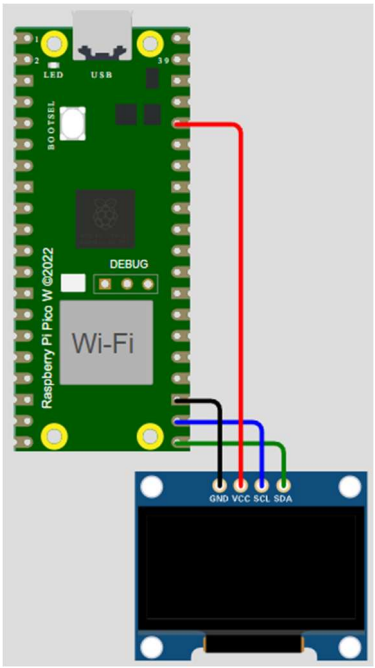


Fig 6: Circuit Diagram for 11

#Code

#main.py

```
from machine import Pin, I2C
from ssd1306 import SSD1306_I2C
import time

WIDTH = 128
HEIGHT = 64

i2c = I2C(0, scl = Pin(17), sda = Pin(16), freq=400000)
display = SSD1306_I2C(WIDTH, HEIGHT, i2c)

def read_temp():
    sensor_temp = machine.ADC(4)
    conversion_factor = 3.3 / (65535)
    reading = sensor_temp.read_u16() * conversion_factor
    temperature = 27 - (reading - 0.706)/0.001721
```

```
    formatted_temperature = "{:.1f}".format(temperature)
    string_temperature = str("Int_Temp:" + formatted_temperature)
    print(string_temperature)
    time.sleep(2)
    return string_temperature

while True:
    display.text('Welcome to ',0,0)
    display.text('AICTE IDEA LAB',0,12)
    display.text('Example 1:', 0,24)
    temperature = read_temp()
    display.text(temperature,0,48)
    display.show()
    display.fill(0)
```

#ssd1306.py

```
# MicroPython SSD1306 OLED driver, I2C and SPI interfaces

from micropython import const
import framebuf

# register definitions
SET_CONTRAST = const(0x81)
SET_ENTIRE_ON = const(0xA4)
SET_NORM_INV = const(0xA6)
SET_DISP = const(0xAE)
SET_MEM_ADDR = const(0x20)
SET_COL_ADDR = const(0x21)
SET_PAGE_ADDR = const(0x22)
SET_DISP_START_LINE = const(0x40)
SET_SEG_REMAP = const(0xA0)
SET_MUX_RATIO = const(0xA8)
SET_IREF_SELECT = const(0xAD)
SET_COM_OUT_DIR = const(0xC0)
SET_DISP_OFFSET = const(0xD3)
SET_COM_PIN_CFG = const(0xDA)
SET_DISP_CLK_DIV = const(0xD5)
SET_PRECHARGE = const(0xD9)
SET_VCOM_DESEL = const(0xDB)
SET_CHARGE_PUMP = const(0x8D)

# Subclassing FrameBuffer provides support for graphics primitives
# http://docs.micropython.org/en/latest/pyboard/library/framebuf.html
class SSD1306(framebuf.FrameBuffer):
    def __init__(self, width, height, external_vcc):
        self.width = width
```

```
        self.height = height
        self.external_vcc = external_vcc
        self.pages = self.height // 8
        self.buffer = bytearray(self.pages * self.width)
        super().__init__(self.buffer, self.width, self.height, framebuf.MONO_VLSB)
        self.init_display()

    def init_display(self):
        for cmd in (
            SET_DISP,  # display off
            # address setting
            SET_MEM_ADDR,
            0x00,  # horizontal
            # resolution and layout
            SET_DISP_START_LINE,  # start at line 0
            SET_SEG_REMAP | 0x01,  # column addr 127 mapped to SEG0
            SET_MUX_RATIO,
            self.height - 1,
            SET_COM_OUT_DIR | 0x08,  # scan from COM[N] to COM0
            SET_DISP_OFFSET,
            0x00,
            SET_COM_PIN_CFG,
            0x02 if self.width > 2 * self.height else 0x12,
            # timing and driving scheme
            SET_DISP_CLK_DIV,
            0x80,
            SET_PRECHARGE,
            0x22 if self.external_vcc else 0xF1,
            SET_VCOM_DESEL,
            0x30,  # 0.83*Vcc
            # display
            SET_CONTRAST,
            0xFF,  # maximum
            SET_ENTIRE_ON,  # output follows RAM contents
            SET_NORM_INV,  # not inverted
            SET_IREF_SELECT,
            0x30,  # enable internal IREF during display on
            # charge pump
            SET_CHARGE_PUMP,
            0x10 if self.external_vcc else 0x14,
            SET_DISP | 0x01,  # display on
        ):  # on
            self.write_cmd(cmd)
        self.fill(0)
        self.show()

    def poweroff(self):
        self.write_cmd(SET_DISP)
```

```
    def poweron(self):
        self.write_cmd(SET_DISP | 0x01)

    def contrast(self, contrast):
        self.write_cmd(SET_CONTRAST)
        self.write_cmd(contrast)

    def invert(self, invert):
        self.write_cmd(SET_NORM_INV | (invert & 1))

    def rotate(self, rotate):
        self.write_cmd(SET_COM_OUT_DIR | ((rotate & 1) << 3))
        self.write_cmd(SET_SEG_REMAP | (rotate & 1))

    def show(self):
        x0 = 0
        x1 = self.width - 1
        if self.width != 128:
            # narrow displays use centred columns
            col_offset = (128 - self.width) // 2
            x0 += col_offset
            x1 += col_offset
        self.write_cmd(SET_COL_ADDR)
        self.write_cmd(x0)
        self.write_cmd(x1)
        self.write_cmd(SET_PAGE_ADDR)
        self.write_cmd(0)
        self.write_cmd(self.pages - 1)
        self.write_data(self.buffer)


class SSD1306_I2C(SSD1306):
    def __init__(self, width, height, i2c, addr=0x3C, external_vcc=False):
        self.i2c = i2c
        self.addr = addr
        self.temp = bytearray(2)
        self.write_list = [b"\x40", None]  # Co=0, D/C#=1
        super().__init__(width, height, external_vcc)

    def write_cmd(self, cmd):
        self.temp[0] = 0x80  # Co=1, D/C#=0
        self.temp[1] = cmd
        self.i2c.writeto(self.addr, self.temp)

    def write_data(self, buf):
        self.write_list[1] = buf
        self.i2c.writevto(self.addr, self.write_list)
```

```
class SSD1306_SPI(SSD1306):
    def __init__(self, width, height, spi, dc, res, cs, external_vcc=False):
        self.rate = 10 * 1024 * 1024
        dc.init(dc.OUT, value=0)
        res.init(res.OUT, value=0)
        cs.init(cs.OUT, value=1)
        self.spi = spi
        self.dc = dc
        self.res = res
        self.cs = cs
        import time

        self.res(1)
        time.sleep_ms(1)
        self.res(0)
        time.sleep_ms(10)
        self.res(1)
        super().__init__(width, height, external_vcc)

    def write_cmd(self, cmd):
        self.spi.init(baudrate=self.rate, polarity=0, phase=0)
        self.cs(1)
        self.dc(0)
        self.cs(0)
        self.spi.write(bytearray([cmd]))
        self.cs(1)

    def write_data(self, buf):
        self.spi.init(baudrate=self.rate, polarity=0, phase=0)
        self.cs(1)
        self.dc(1)
        self.cs(0)
        self.spi.write(buf)
        self.cs(1)
```

Experiment 12: Interface 7 segment LED display with Pico W to display numbers 0-9 in ascending and descending order

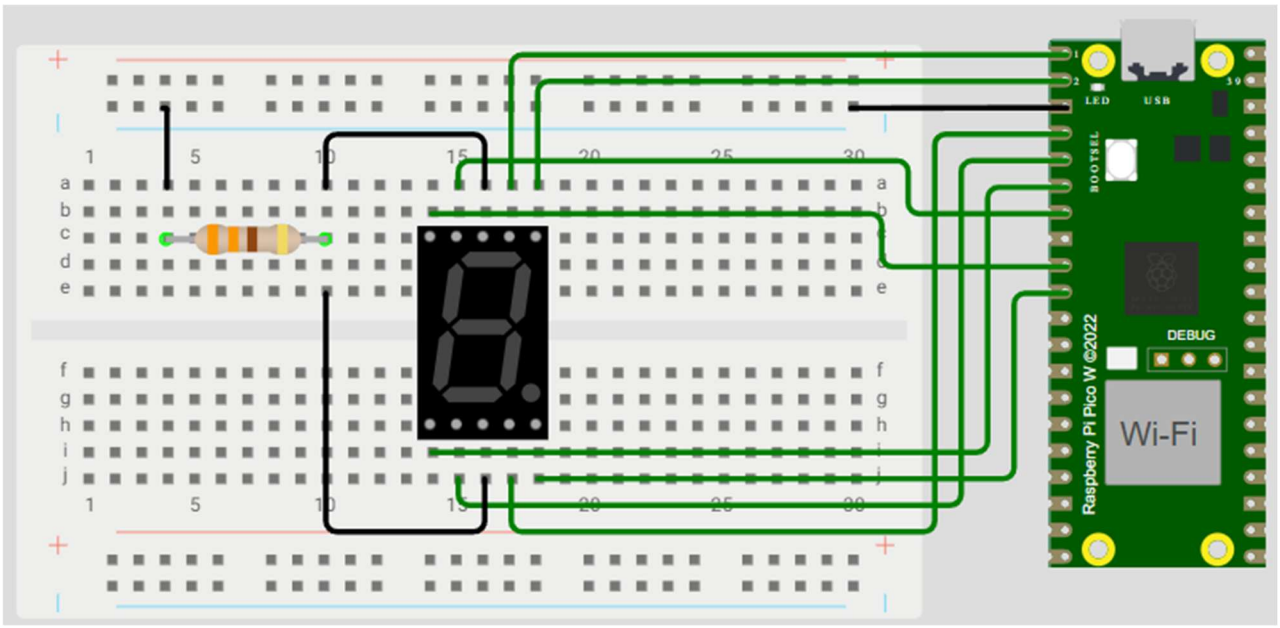


Fig 7: Circuit Diagram for 12

#Code

#main.py

```
from machine import Pin
import utime

pin_list = [0,1,2,3,4,5,6,7]
L = [0]*8

for i in range (8):
 L[i] = Pin(pin_list[i],Pin.OUT)

#Common Cathode display
chars = [
    [1,1,1,1,1,1,0,0], #0
    [0,1,1,0,0,0,0,0], #1
    [1,1,0,1,1,0,1,0], #2
    [1,1,1,1,0,0,1,0], #3
    [0,1,1,0,0,1,1,0], #4
    [1,0,1,1,0,1,1,0], #5
    [1,0,1,1,1,1,1,0], #6
    [1,1,1,0,0,0,0,0], #7
    [1,1,1,1,1,1,1,0], #8
    [1,1,1,1,0,1,1,0], #9
    [0,0,0,0,0,0,0,1] #dot
```

```
]

for i in pin_list:
    L[i].value(0)

while True:
    for i in range(len(chars)):
        for j in range(len(pin_list)):
            L[j].value(chars[i][j])
        utime.sleep(1)
    for i in range(len(chars)-1,-1,-1):
        for j in range(len(pin_list)):
            L[j].value(chars[i][j])
        utime.sleep(1)
    utime.sleep(0.5)
```

Experiment 13: Interface programmable LED strip (WS2812B) with Pico W

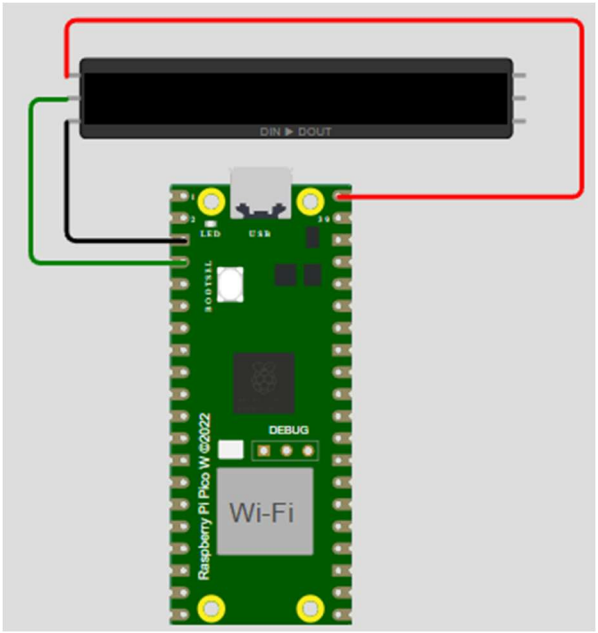


Fig 8: Circuit diagram for 13

#Code

#main.py

```
from neopixel import Neopixel
import utime
import random

numpix = 8
strip = Neopixel(numpix, 0, 2, "RGB")

red = (255, 0, 0)
orange = (255, 50, 0)
yellow = (255, 100, 0)
green = (0, 255, 0)
blue = (0, 0, 255)
indigo = (100, 0, 90)
violet = (200, 0, 100)
colors_rgb = [red, orange, yellow, green, blue, indigo, violet]

delay = 0.5
strip.brightness(200)
blank = (0,0,0)

while True:
    #Controlling individual leds
    strip.set_pixel(random.randint(0, numpix-1), colors_rgb[random.randint(0,
len(colors_rgb)-1)])
    strip.set_pixel(random.randint(0, numpix-1), colors_rgb[random.randint(0,
len(colors_rgb)-1)])
    strip.set_pixel(random.randint(0, numpix-1), colors_rgb[random.randint(0,
len(colors_rgb)-1)])
    strip.set_pixel(random.randint(0, numpix-1), colors_rgb[random.randint(0,
len(colors_rgb)-1)])
    strip.set_pixel(random.randint(0, numpix-1), colors_rgb[random.randint(0,
len(colors_rgb)-1)])
    strip.set_pixel(random.randint(0, numpix-1), colors_rgb[random.randint(0,
len(colors_rgb)-1)])
    strip.set_pixel(random.randint(0, numpix-1), colors_rgb[random.randint(0,
len(colors_rgb)-1)])
    strip.set_pixel(random.randint(0, numpix-1), colors_rgb[random.randint(0,
len(colors_rgb)-1)])
    strip.show()
    utime.sleep(delay)
    strip.fill((0,0,0))
```

#neopixel.py

```
import array, time
from machine import Pin
import rp2

# PIO state machine for RGB. Pulls 24 bits (rgb -> 3 * 8bit) automatically
@rp2.asm_pio(sideset_init=rp2.PIO.OUT_LOW, out_shiftdir=rp2.PIO.SHIFT_LEFT,
autopull=True, pull_thresh=24)
def ws2812():
    T1 = 2
    T2 = 5
    T3 = 3
    wrap_target()
    label("bitloop")
    out(x, 1)               .side(0)    [T3 - 1]
    jmp(not_x, "do_zero")   .side(1)    [T1 - 1]
    jmp("bitloop")          .side(1)    [T2 - 1]
    label("do_zero")
    nop().side(0)                       [T2 - 1]
    wrap()

# PIO state machine for RGBW. Pulls 32 bits (rgbw -> 4 * 8bit) automatically
@rp2.asm_pio(sideset_init=rp2.PIO.OUT_LOW, out_shiftdir=rp2.PIO.SHIFT_LEFT,
autopull=True, pull_thresh=32)
def sk6812():
    T1 = 2
    T2 = 5
    T3 = 3
    wrap_target()
    label("bitloop")
    out(x, 1)               .side(0)    [T3 - 1]
    jmp(not_x, "do_zero")   .side(1)    [T1 - 1]
    jmp("bitloop")          .side(1)    [T2 - 1]
    label("do_zero")
    nop()                   .side(0)    [T2 - 1]
    wrap()


# Delay here is the reset time. You need a pause to reset the LED strip back to the
initial LED
# however, if you have quite a bit of processing to do before the next time you update
the strip
# you could put in delay=0 (or a lower delay)
#
# Class supports different order of individual colors (GRB, RGB, WRGB, GWRB ...). In
order to achieve
# this, we need to flip the indexes: in 'RGBW', 'R' is on index 0, but we need to shift
it left by 3 * 8bits,
```

```
# so in it's inverse, 'WBGR', it has exactly right index. Since micropython doesn't have
[::-1] and recursive rev()
# isn't too efficient we simply do that by XORing (operator ^) each index with 3 (0b11)
to make this flip.
# When dealing with just 'RGB' (3 letter string), this means same but reduced by 1 after
XOR!.
# Example: in 'GRBW' we want final form of 0bGGRRBBWW, meaning G with index 0 needs to
be shifted 3 * 8bit ->
# 'G' on index 0: 0b00 ^ 0b11 -> 0b11 (3), just as we wanted.
# Same hold for every other index (and - 1 at the end for 3 letter strings).

class Neopixel:
    def __init__(self, num_leds, state_machine, pin, mode="RGB", delay=0.0001):
        self.pixels = array.array("I", [0 for _ in range(num_leds)])
        self.mode = set(mode)   # set for better performance
        if 'W' in self.mode:
            # RGBW uses different PIO state machine configuration
            self.sm = rp2.StateMachine(state_machine, sk6812, freq=8000000,
sideset_base=Pin(pin))
            # dictionary of values required to shift bit into position (check class
desc.)
            self.shift = {'R': (mode.index('R') ^ 3) * 8, 'G': (mode.index('G') ^ 3) *
8,
                          'B': (mode.index('B') ^ 3) * 8, 'W': (mode.index('W') ^ 3) *
8}
        else:
            self.sm = rp2.StateMachine(state_machine, ws2812, freq=8000000,
sideset_base=Pin(pin))
            self.shift = {'R': ((mode.index('R') ^ 3) - 1) * 8, 'G': ((mode.index('G') ^
3) - 1) * 8,
                          'B': ((mode.index('B') ^ 3) - 1) * 8, 'W': 0}
        self.sm.active(1)
        self.num_leds = num_leds
        self.delay = delay
        self.brightnessvalue = 255

    # Set the overal value to adjust brightness when updating leds
    def brightness(self, brightness=None):
        if brightness == None:
            return self.brightnessvalue
        else:
            if brightness < 1:
                brightness = 1
        if brightness > 255:
            brightness = 255
        self.brightnessvalue = brightness

    # Create a gradient with two RGB colors between "pixel1" and "pixel2" (inclusive)
```

```
    # Function accepts two (r, g, b) / (r, g, b, w) tuples
    def set_pixel_line_gradient(self, pixel1, pixel2, left_rgb_w, right_rgb_w):
        if pixel2 - pixel1 == 0:
            return
        right_pixel = max(pixel1, pixel2)
        left_pixel = min(pixel1, pixel2)

        for i in range(right_pixel - left_pixel + 1):
            fraction = i / (right_pixel - left_pixel)
            red = round((right_rgb_w[0] - left_rgb_w[0]) * fraction + left_rgb_w[0])
            green = round((right_rgb_w[1] - left_rgb_w[1]) * fraction + left_rgb_w[1])
            blue = round((right_rgb_w[2] - left_rgb_w[2]) * fraction + left_rgb_w[2])
            # if it's (r, g, b, w)
            if len(left_rgb_w) == 4 and 'W' in self.mode:
                white = round((right_rgb_w[3] - left_rgb_w[3]) * fraction +
left_rgb_w[3])
                self.set_pixel(left_pixel + i, (red, green, blue, white))
            else:
                self.set_pixel(left_pixel + i, (red, green, blue))

    # Set an array of pixels starting from "pixel1" to "pixel2" (inclusive) to the
desired color.
    # Function accepts (r, g, b) / (r, g, b, w) tuple
    def set_pixel_line(self, pixel1, pixel2, rgb_w):
        for i in range(pixel1, pixel2 + 1):
            self.set_pixel(i, rgb_w)

    # Set red, green and blue value of pixel on position <pixel_num>
    # Function accepts (r, g, b) / (r, g, b, w) tuple
    def set_pixel(self, pixel_num, rgb_w):
        pos = self.shift

        red = round(rgb_w[0] * (self.brightness() / 255))
        green = round(rgb_w[1] * (self.brightness() / 255))
        blue = round(rgb_w[2] * (self.brightness() / 255))
        white = 0
        # if it's (r, g, b, w)
        if len(rgb_w) == 4 and 'W' in self.mode:
            white = round(rgb_w[3] * (self.brightness() / 255))

        self.pixels[pixel_num] = white << pos['W'] | blue << pos['B'] | red << pos['R']
| green << pos['G']

    # Converts HSV color to rgb tuple and returns it
    # Function accepts integer values for <hue>, <saturation> and <value>
    # The logic is almost the same as in Adafruit NeoPixel library:
    # https://github.com/adafruit/Adafruit_NeoPixel so all the credits for that
```

```
    # go directly to them (license:
https://github.com/adafruit/Adafruit_NeoPixel/blob/master/COPYING)
    def colorHSV(self, hue, sat, val):
        if hue >= 65536:
            hue %= 65536

        hue = (hue * 1530 + 32768) // 65536
        if hue < 510:
            b = 0
            if hue < 255:
                r = 255
                g = hue
            else:
                r = 510 - hue
                g = 255
        elif hue < 1020:
            r = 0
            if hue < 765:
                g = 255
                b = hue - 510
            else:
                g = 1020 - hue
                b = 255
        elif hue < 1530:
            g = 0
            if hue < 1275:
                r = hue - 1020
                b = 255
            else:
                r = 255
                b = 1530 - hue
        else:
            r = 255
            g = 0
            b = 0

        v1 = 1 + val
        s1 = 1 + sat
        s2 = 255 - sat

        r = ((((r * s1) >> 8) + s2) * v1) >> 8
        g = ((((g * s1) >> 8) + s2) * v1) >> 8
        b = ((((b * s1) >> 8) + s2) * v1) >> 8

        return r, g, b


    # Rotate <num_of_pixels> pixels to the left
    def rotate_left(self, num_of_pixels):
```

```
        if num_of_pixels == None:
            num_of_pixels = 1
        self.pixels = self.pixels[num_of_pixels:] + self.pixels[:num_of_pixels]

    # Rotate <num_of_pixels> pixels to the right
    def rotate_right(self, num_of_pixels):
        if num_of_pixels == None:
            num_of_pixels = 1
        num_of_pixels = -1 * num_of_pixels
        self.pixels = self.pixels[num_of_pixels:] + self.pixels[:num_of_pixels]

    # Update pixels
    def show(self):
        # If mode is RGB, we cut 8 bits of, otherwise we keep all 32
        cut = 8
        if 'W' in self.mode:
            cut = 0
        for i in range(self.num_leds):
            self.sm.put(self.pixels[i], cut)
        time.sleep(self.delay)

    # Set all pixels to given rgb values
    # Function accepts (r, g, b) / (r, g, b, w)
    def fill(self, rgb_w):
        for i in range(self.num_leds):
            self.set_pixel(i, rgb_w)
        time.sleep(self.delay)
```

Experiment 14: Interface ILI9341 2.8inch TFT LCD touchscreen display with Pico W

Ref: https://diyprojectslabs.com/raspberry-pi-pico-tft-lcd-touch-screen-tutorial/

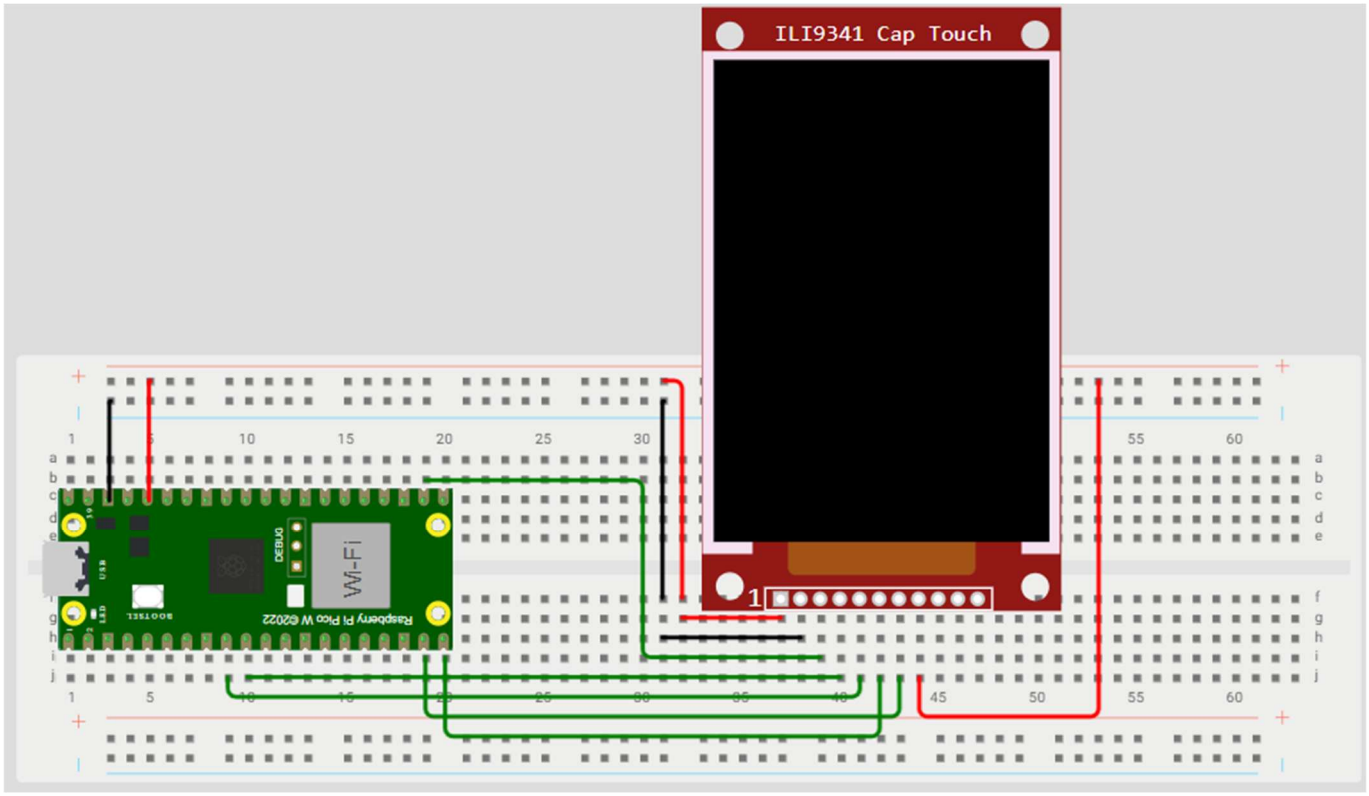


Fig 9: Circuit diagram for 14

#Code

#Example 1

#colours.py

```
'''
from time import sleep
from ili9341 import Display, color565
from machine import Pin, SPI


def test():
    """Test code."""
    # Baud rate of 40000000 seems about the max
    spi = SPI(1, baudrate=10000000, sck=Pin(14), mosi=Pin(15))
    display = Display(spi, dc=Pin(6), cs=Pin(17), rst=Pin(7))

    # Set display color to red
    display.clear(color565(255, 0, 0))
```

```
    sleep(1)

    # Set display color to orange
    display.clear(color565(255, 128, 0))
    sleep(1)

    # Set display color to yellow
    display.clear(color565(255, 255, 0))
    sleep(1)

    # Set display color to green
    display.clear(color565(0, 255, 0))
    sleep(1)

    # Set display color to blue
    display.clear(color565(0, 0, 255))
    sleep(1)

    # Set display color to purple
    display.clear(color565(128, 0, 128))
    sleep(1)

    # Set display color to pink
    display.clear(color565(255, 192, 203))
    sleep(1)

    # Set display color to brown
    display.clear(color565(139, 69, 19))
    sleep(1)

    # Set display color to gray
    display.clear(color565(128, 128, 128))
    sleep(1)

    # Set display color to white
    display.clear(color565(255, 255, 255))
    sleep(1)

test()
'''
```

#Example2

#animation.py

```
'''
from time import sleep
from ili9341 import Display, color565
from machine import Pin, SPI
```

```
# Constants
SPI_SPEED = 10000000
DC_PIN = Pin(6)
CS_PIN = Pin(17)
RST_PIN = Pin(7)
BACK_COLOR = color565(0, 0, 0)
RECT_COLOR = color565(255, 0, 0)
POLY_COLOR = color565(0, 64, 255)
CIRC_COLOR = color565(0, 255, 0)
ELLP_COLOR = color565(255, 0, 0)

# Create display object
spi = SPI(1, baudrate=SPI_SPEED, sck=Pin(14), mosi=Pin(15))
display = Display(spi, dc=DC_PIN, cs=CS_PIN, rst=RST_PIN)

# Draw rectangles
def draw_rectangles():
    for x in range(0, 225, 15):
        display.fill_rectangle(x, 0, 15, 227, RECT_COLOR)

# Draw polygons
def draw_polygons():
    display.fill_polygon(7, 120, 120, 100, POLY_COLOR)
    sleep(1)
    display.draw_polygon(3, 120, 286, 30, POLY_COLOR, rotate=15)
    sleep(3)

# Draw circles
def draw_circles():
    display.fill_circle(132, 132, 70, CIRC_COLOR)
    sleep(1)
    display.draw_circle(132, 96, 70, color565(0, 0, 255))
    sleep(1)

# Draw ellipses
def draw_ellipses():
    display.fill_ellipse(96, 96, 30, 16, ELLP_COLOR)
    sleep(1)
    display.draw_ellipse(96, 256, 16, 30, color565(255, 255, 0))

# Clear screen and draw shapes
display.clear(BACK_COLOR)
draw_rectangles()
display.clear(BACK_COLOR)
draw_polygons()
display.clear(BACK_COLOR)
draw_circles()
```

```
display.clear(BACK_COLOR)
draw_ellipses()

# Clean up
display.cleanup()
'''
```

#Example 3

#bouncing_box.py

```
from machine import Pin, SPI
from random import random, seed
from ili9341 import Display, color565
from utime import sleep_us, ticks_cpu, ticks_us, ticks_diff


class Box(object):
    """Bouncing box."""

    def __init__(self, screen_width, screen_height, size, display, color):
        """Initialize box.

        Args:
            screen_width (int): Width of screen.
            screen_height (int): Width of height.
            size (int): Square side length.
            display (ILI9341): display object.
            color (int): RGB565 color value.
        """
        self.size = size
        self.w = screen_width
        self.h = screen_height
        self.display = display
        self.color = color
        # Generate non-zero random speeds between -5.0 and 5.0
        seed(ticks_cpu())
        r = random() * 10.0
        self.x_speed = 5.0 - r if r < 5.0 else r - 10.0
        r = random() * 10.0
        self.y_speed = 5.0 - r if r < 5.0 else r - 10.0

        self.x = self.w / 2.0
        self.y = self.h / 2.0
        self.prev_x = self.x
        self.prev_y = self.y

    def update_pos(self):
```

```
        """Update box position and speed."""
        x = self.x
        y = self.y
        size = self.size
        w = self.w
        h = self.h
        x_speed = abs(self.x_speed)
        y_speed = abs(self.y_speed)
        self.prev_x = x
        self.prev_y = y

        if x + size >= w - x_speed:
            self.x_speed = -x_speed
        elif x - size <= x_speed + 1:
            self.x_speed = x_speed

        if y + size >= h - y_speed:
            self.y_speed = -y_speed
        elif y - size <= y_speed + 1:
            self.y_speed = y_speed

        self.x = x + self.x_speed
        self.y = y + self.y_speed

    def draw(self):
        """Draw box."""
        x = int(self.x)
        y = int(self.y)
        size = self.size
        prev_x = int(self.prev_x)
        prev_y = int(self.prev_y)
        self.display.fill_hrect(prev_x - size,
                                prev_y - size,
                                size, size, 0)
        self.display.fill_hrect(x - size,
                                y - size,
                                size, size, self.color)


def test():
    """Bouncing box."""
    try:
        # Baud rate of 40000000 seems about the max
        spi = SPI(1, baudrate=10000000, sck=Pin(14), mosi=Pin(15))
        display = Display(spi, dc=Pin(6), cs=Pin(17), rst=Pin(7))
        display.clear()

        colors = [color565(255, 0, 0),
                  color565(0, 255, 0),
```

```
                  color565(0, 0, 255),
                  color565(255, 255, 0),
                  color565(0, 255, 255),
                  color565(255, 0, 255)]
        sizes = [12, 11, 10, 9, 8, 7]
        boxes = [Box(239, 319, sizes[i], display,
                 colors[i]) for i in range(6)]

        while True:
            timer = ticks_us()
            for b in boxes:
                b.update_pos()
                b.draw()
            # Attempt to set framerate to 30 FPS
            timer_dif = 33333 - ticks_diff(ticks_us(), timer)
            if timer_dif > 0:
                sleep_us(timer_dif)

    except KeyboardInterrupt:
        display.cleanup()


test()
```

#ili9341.py

```
"""ILI9341 LCD/Touch module."""
from time import sleep
from math import cos, sin, pi, radians
from sys import implementation
from framebuf import FrameBuffer, RGB565  # type: ignore
import ustruct  # type: ignore


def color565(r, g, b):
    """Return RGB565 color value.

    Args:
        r (int): Red value.
        g (int): Green value.
        b (int): Blue value.
    """
    return (r & 0xf8) << 8 | (g & 0xfc) << 3 | b >> 3
```

```
class Display(object):
    """Serial interface for 16-bit color (5-6-5 RGB) ILI9341 display.
    Note:  All coordinates are zero based.
    """
    # Command constants from ILI9341 datasheet
    NOP = const(0x00)  # No-op
    SWRESET = const(0x01)  # Software reset
    RDDID = const(0x04)  # Read display ID info
    RDDST = const(0x09)  # Read display status
    SLPIN = const(0x10)  # Enter sleep mode
    SLPOUT = const(0x11)  # Exit sleep mode
    PTLON = const(0x12)  # Partial mode on
    NORON = const(0x13)  # Normal display mode on
    RDMODE = const(0x0A)  # Read display power mode
    RDMADCTL = const(0x0B)  # Read display MADCTL
    RDPIXFMT = const(0x0C)  # Read display pixel format
    RDIMGFMT = const(0x0D)  # Read display image format
    RDSELFDIAG = const(0x0F)  # Read display self-diagnostic
    INVOFF = const(0x20)  # Display inversion off
    INVON = const(0x21)  # Display inversion on
    GAMMASET = const(0x26)  # Gamma set
    DISPLAY_OFF = const(0x28)  # Display off
    DISPLAY_ON = const(0x29)  # Display on
    SET_COLUMN = const(0x2A)  # Column address set
    SET_PAGE = const(0x2B)  # Page address set
    WRITE_RAM = const(0x2C)  # Memory write
    READ_RAM = const(0x2E)  # Memory read
    PTLAR = const(0x30)  # Partial area
    VSCRDEF = const(0x33)  # Vertical scrolling definition
    MADCTL = const(0x36)  # Memory access control
    VSCRSADD = const(0x37)  # Vertical scrolling start address
    PIXFMT = const(0x3A)  # COLMOD: Pixel format set
    WRITE_DISPLAY_BRIGHTNESS = const(0x51)  # Brightness hardware dependent!
    READ_DISPLAY_BRIGHTNESS = const(0x52)
    WRITE_CTRL_DISPLAY = const(0x53)
    READ_CTRL_DISPLAY = const(0x54)
    WRITE_CABC = const(0x55)  # Write Content Adaptive Brightness Control
    READ_CABC = const(0x56)  # Read Content Adaptive Brightness Control
    WRITE_CABC_MINIMUM = const(0x5E)  # Write CABC Minimum Brightness
    READ_CABC_MINIMUM = const(0x5F)  # Read CABC Minimum Brightness
    FRMCTR1 = const(0xB1)  # Frame rate control (In normal mode/full colors)
    FRMCTR2 = const(0xB2)  # Frame rate control (In idle mode/8 colors)
    FRMCTR3 = const(0xB3)  # Frame rate control (In partial mode/full colors)
    INVCTR = const(0xB4)  # Display inversion control
    DFUNCTR = const(0xB6)  # Display function control
    PWCTR1 = const(0xC0)  # Power control 1
    PWCTR2 = const(0xC1)  # Power control 2
    PWCTRA = const(0xCB)  # Power control A
```

```
PWCTRB = const(0xCF)  # Power control B
VMCTR1 = const(0xC5)  # VCOM control 1
VMCTR2 = const(0xC7)  # VCOM control 2
RDID1 = const(0xDA)  # Read ID 1
RDID2 = const(0xDB)  # Read ID 2
RDID3 = const(0xDC)  # Read ID 3
RDID4 = const(0xDD)  # Read ID 4
GMCTRP1 = const(0xE0)  # Positive gamma correction
GMCTRN1 = const(0xE1)  # Negative gamma correction
DTCA = const(0xE8)  # Driver timing control A
DTCB = const(0xEA)  # Driver timing control B
POSC = const(0xED)  # Power on sequence control
ENABLE3G = const(0xF2)  # Enable 3 gamma control
PUMPRC = const(0xF7)  # Pump ratio control

ROTATE = {
    0: 0x88,
    90: 0xE8,
    180: 0x48,
    270: 0x28
}

def __init__(self, spi, cs, dc, rst,
             width=240, height=320, rotation=0):
    """Initialize OLED.

    Args:
        spi (Class Spi):  SPI interface for OLED
        cs (Class Pin):  Chip select pin
        dc (Class Pin):  Data/Command pin
        rst (Class Pin):  Reset pin
        width (Optional int): Screen width (default 240)
        height (Optional int): Screen height (default 320)
        rotation (Optional int): Rotation must be 0 default, 90. 180 or 270
    """
    self.spi = spi
    self.cs = cs
    self.dc = dc
    self.rst = rst
    self.width = width
    self.height = height
    if rotation not in self.ROTATE.keys():
        raise RuntimeError('Rotation must be 0, 90, 180 or 270.')
    else:
        self.rotation = self.ROTATE[rotation]

    # Initialize GPIO pins and set implementation specific methods
    if implementation.name == 'circuitpython':
```

```
            self.cs.switch_to_output(value=True)
            self.dc.switch_to_output(value=False)
            self.rst.switch_to_output(value=True)
            self.reset = self.reset_cpy
            self.write_cmd = self.write_cmd_cpy
            self.write_data = self.write_data_cpy
        else:
            self.cs.init(self.cs.OUT, value=1)
            self.dc.init(self.dc.OUT, value=0)
            self.rst.init(self.rst.OUT, value=1)
            self.reset = self.reset_mpy
            self.write_cmd = self.write_cmd_mpy
            self.write_data = self.write_data_mpy
        self.reset()
        # Send initialization commands
        self.write_cmd(self.SWRESET)  # Software reset
        sleep(.1)
        self.write_cmd(self.PWCTRB, 0x00, 0xC1, 0x30)  # Pwr ctrl B
        self.write_cmd(self.POSC, 0x64, 0x03, 0x12, 0x81)  # Pwr on seq. ctrl
        self.write_cmd(self.DTCA, 0x85, 0x00, 0x78)  # Driver timing ctrl A
        self.write_cmd(self.PWCTRA, 0x39, 0x2C, 0x00, 0x34, 0x02)  # Pwr ctrl A
        self.write_cmd(self.PUMPRC, 0x20)  # Pump ratio control
        self.write_cmd(self.DTCB, 0x00, 0x00)  # Driver timing ctrl B
        self.write_cmd(self.PWCTR1, 0x23)  # Pwr ctrl 1
        self.write_cmd(self.PWCTR2, 0x10)  # Pwr ctrl 2
        self.write_cmd(self.VMCTR1, 0x3E, 0x28)  # VCOM ctrl 1
        self.write_cmd(self.VMCTR2, 0x86)  # VCOM ctrl 2
        self.write_cmd(self.MADCTL, self.rotation)  # Memory access ctrl
        self.write_cmd(self.VSCRSADD, 0x00)  # Vertical scrolling start address
        self.write_cmd(self.PIXFMT, 0x55)  # COLMOD: Pixel format
        self.write_cmd(self.FRMCTR1, 0x00, 0x18)  # Frame rate ctrl
        self.write_cmd(self.DFUNCTR, 0x08, 0x82, 0x27)
        self.write_cmd(self.ENABLE3G, 0x00)  # Enable 3 gamma ctrl
        self.write_cmd(self.GAMMASET, 0x01)  # Gamma curve selected
        self.write_cmd(self.GMCTRP1, 0x0F, 0x31, 0x2B, 0x0C, 0x0E, 0x08, 0x4E,
                       0xF1, 0x37, 0x07, 0x10, 0x03, 0x0E, 0x09, 0x00)
        self.write_cmd(self.GMCTRN1, 0x00, 0x0E, 0x14, 0x03, 0x11, 0x07, 0x31,
                       0xC1, 0x48, 0x08, 0x0F, 0x0C, 0x31, 0x36, 0x0F)
        self.write_cmd(self.SLPOUT)  # Exit sleep
        sleep(.1)
        self.write_cmd(self.DISPLAY_ON)  # Display on
        sleep(.1)
        self.clear()

    def block(self, x0, y0, x1, y1, data):
        """Write a block of data to display.

        Args:
```

```
            x0 (int):  Starting X position.
            y0 (int):  Starting Y position.
            x1 (int):  Ending X position.
            y1 (int):  Ending Y position.
            data (bytes): Data buffer to write.
        """
        self.write_cmd(self.SET_COLUMN, *ustruct.pack(">HH", x0, x1))
        self.write_cmd(self.SET_PAGE, *ustruct.pack(">HH", y0, y1))

        self.write_cmd(self.WRITE_RAM)
        self.write_data(data)

    def cleanup(self):
        """Clean up resources."""
        self.clear()
        self.display_off()
        self.spi.deinit()
        print('display off')

    def clear(self, color=0):
        """Clear display.

        Args:
            color (Optional int): RGB565 color value (Default: 0 = Black).
        """
        w = self.width
        h = self.height
        # Clear display in 1024 byte blocks
        if color:
            line = color.to_bytes(2, 'big') * (w * 8)
        else:
            line = bytearray(w * 16)
        for y in range(0, h, 8):
            self.block(0, y, w - 1, y + 7, line)

    def display_off(self):
        """Turn display off."""
        self.write_cmd(self.DISPLAY_OFF)

    def display_on(self):
        """Turn display on."""
        self.write_cmd(self.DISPLAY_ON)

    def draw_circle(self, x0, y0, r, color):
        """Draw a circle.

        Args:
            x0 (int): X coordinate of center point.
```

```
            y0 (int): Y coordinate of center point.
            r (int): Radius.
            color (int): RGB565 color value.
        """
        f = 1 - r
        dx = 1
        dy = -r - r
        x = 0
        y = r
        self.draw_pixel(x0, y0 + r, color)
        self.draw_pixel(x0, y0 - r, color)
        self.draw_pixel(x0 + r, y0, color)
        self.draw_pixel(x0 - r, y0, color)
        while x < y:
            if f >= 0:
                y -= 1
                dy += 2
                f += dy
            x += 1
            dx += 2
            f += dx
            self.draw_pixel(x0 + x, y0 + y, color)
            self.draw_pixel(x0 - x, y0 + y, color)
            self.draw_pixel(x0 + x, y0 - y, color)
            self.draw_pixel(x0 - x, y0 - y, color)
            self.draw_pixel(x0 + y, y0 + x, color)
            self.draw_pixel(x0 - y, y0 + x, color)
            self.draw_pixel(x0 + y, y0 - x, color)
            self.draw_pixel(x0 - y, y0 - x, color)

    def draw_ellipse(self, x0, y0, a, b, color):
        """Draw an ellipse.

        Args:
            x0, y0 (int): Coordinates of center point.
            a (int): Semi axis horizontal.
            b (int): Semi axis vertical.
            color (int): RGB565 color value.
        Note:
            The center point is the center of the x0,y0 pixel.
            Since pixels are not divisible, the axes are integer rounded
            up to complete on a full pixel.  Therefore the major and
            minor axes are increased by 1.
        """
        a2 = a * a
        b2 = b * b
        twoa2 = a2 + a2
        twob2 = b2 + b2
```

```
        x = 0
        y = b
        px = 0
        py = twoa2 * y
        # Plot initial points
        self.draw_pixel(x0 + x, y0 + y, color)
        self.draw_pixel(x0 - x, y0 + y, color)
        self.draw_pixel(x0 + x, y0 - y, color)
        self.draw_pixel(x0 - x, y0 - y, color)
        # Region 1
        p = round(b2 - (a2 * b) + (0.25 * a2))
        while px < py:
            x += 1
            px += twob2
            if p < 0:
                p += b2 + px
            else:
                y -= 1
                py -= twoa2
                p += b2 + px - py
            self.draw_pixel(x0 + x, y0 + y, color)
            self.draw_pixel(x0 - x, y0 + y, color)
            self.draw_pixel(x0 + x, y0 - y, color)
            self.draw_pixel(x0 - x, y0 - y, color)
        # Region 2
        p = round(b2 * (x + 0.5) * (x + 0.5) +
                  a2 * (y - 1) * (y - 1) - a2 * b2)
        while y > 0:
            y -= 1
            py -= twoa2
            if p > 0:
                p += a2 - py
            else:
                x += 1
                px += twob2
                p += a2 - py + px
            self.draw_pixel(x0 + x, y0 + y, color)
            self.draw_pixel(x0 - x, y0 + y, color)
            self.draw_pixel(x0 + x, y0 - y, color)
            self.draw_pixel(x0 - x, y0 - y, color)

    def draw_hline(self, x, y, w, color):
        """Draw a horizontal line.

        Args:
            x (int): Starting X position.
            y (int): Starting Y position.
            w (int): Width of line.
```

```
            color (int): RGB565 color value.
        """
        if self.is_off_grid(x, y, x + w - 1, y):
            return
        line = color.to_bytes(2, 'big') * w
        self.block(x, y, x + w - 1, y, line)

    def draw_image(self, path, x=0, y=0, w=320, h=240):
        """Draw image from flash.

        Args:
            path (string): Image file path.
            x (int): X coordinate of image left.  Default is 0.
            y (int): Y coordinate of image top.  Default is 0.
            w (int): Width of image.  Default is 320.
            h (int): Height of image.  Default is 240.
        """
        x2 = x + w - 1
        y2 = y + h - 1
        if self.is_off_grid(x, y, x2, y2):
            return
        with open(path, "rb") as f:
            chunk_height = 1024 // w
            chunk_count, remainder = divmod(h, chunk_height)
            chunk_size = chunk_height * w * 2
            chunk_y = y
            if chunk_count:
                for c in range(0, chunk_count):
                    buf = f.read(chunk_size)
                    self.block(x, chunk_y,
                               x2, chunk_y + chunk_height - 1,
                               buf)
                    chunk_y += chunk_height
            if remainder:
                buf = f.read(remainder * w * 2)
                self.block(x, chunk_y,
                           x2, chunk_y + remainder - 1,
                           buf)

    def draw_letter(self, x, y, letter, font, color, background=0,
                    landscape=False):
        """Draw a letter.

        Args:
            x (int): Starting X position.
            y (int): Starting Y position.
            letter (string): Letter to draw.
            font (XglcdFont object): Font.
```

```
            color (int): RGB565 color value.
            background (int): RGB565 background color (default: black).
            landscape (bool): Orientation (default: False = portrait)
        """
        buf, w, h = font.get_letter(letter, color, background, landscape)
        # Check for errors (Font could be missing specified letter)
        if w == 0:
            return w, h

        if landscape:
            y -= w
            if self.is_off_grid(x, y, x + h - 1, y + w - 1):
                return 0, 0
            self.block(x, y,
                       x + h - 1, y + w - 1,
                       buf)
        else:
            if self.is_off_grid(x, y, x + w - 1, y + h - 1):
                return 0, 0
            self.block(x, y,
                       x + w - 1, y + h - 1,
                       buf)
        return w, h

    def draw_line(self, x1, y1, x2, y2, color):
        """Draw a line using Bresenham's algorithm.

        Args:
            x1, y1 (int): Starting coordinates of the line
            x2, y2 (int): Ending coordinates of the line
            color (int): RGB565 color value.
        """
        # Check for horizontal line
        if y1 == y2:
            if x1 > x2:
                x1, x2 = x2, x1
            self.draw_hline(x1, y1, x2 - x1 + 1, color)
            return
        # Check for vertical line
        if x1 == x2:
            if y1 > y2:
                y1, y2 = y2, y1
            self.draw_vline(x1, y1, y2 - y1 + 1, color)
            return
        # Confirm coordinates in boundary
        if self.is_off_grid(min(x1, x2), min(y1, y2),
                            max(x1, x2), max(y1, y2)):
            return
```

```
        # Changes in x, y
        dx = x2 - x1
        dy = y2 - y1
        # Determine how steep the line is
        is_steep = abs(dy) > abs(dx)
        # Rotate line
        if is_steep:
            x1, y1 = y1, x1
            x2, y2 = y2, x2
        # Swap start and end points if necessary
        if x1 > x2:
            x1, x2 = x2, x1
            y1, y2 = y2, y1
        # Recalculate differentials
        dx = x2 - x1
        dy = y2 - y1
        # Calculate error
        error = dx >> 1
        ystep = 1 if y1 < y2 else -1
        y = y1
        for x in range(x1, x2 + 1):
            # Had to reverse HW ????
            if not is_steep:
                self.draw_pixel(x, y, color)
            else:
                self.draw_pixel(y, x, color)
            error -= abs(dy)
            if error < 0:
                y += ystep
                error += dx

    def draw_lines(self, coords, color):
        """Draw multiple lines.

        Args:
            coords ([[int, int],...]): Line coordinate X, Y pairs
            color (int): RGB565 color value.
        """
        # Starting point
        x1, y1 = coords[0]
        # Iterate through coordinates
        for i in range(1, len(coords)):
            x2, y2 = coords[i]
            self.draw_line(x1, y1, x2, y2, color)
            x1, y1 = x2, y2

    def draw_pixel(self, x, y, color):
        """Draw a single pixel.
```

```
        Args:
            x (int): X position.
            y (int): Y position.
            color (int): RGB565 color value.
        """
        if self.is_off_grid(x, y, x, y):
            return
        self.block(x, y, x, y, color.to_bytes(2, 'big'))

    def draw_polygon(self, sides, x0, y0, r, color, rotate=0):
        """Draw an n-sided regular polygon.

        Args:
            sides (int): Number of polygon sides.
            x0, y0 (int): Coordinates of center point.
            r (int): Radius.
            color (int): RGB565 color value.
            rotate (Optional float): Rotation in degrees relative to origin.
        Note:
            The center point is the center of the x0,y0 pixel.
            Since pixels are not divisible, the radius is integer rounded
            up to complete on a full pixel.  Therefore diameter = 2 x r + 1.
        """
        coords = []
        theta = radians(rotate)
        n = sides + 1
        for s in range(n):
            t = 2.0 * pi * s / sides + theta
            coords.append([int(r * cos(t) + x0), int(r * sin(t) + y0)])

        # Cast to python float first to fix rounding errors
        self.draw_lines(coords, color=color)

    def draw_rectangle(self, x, y, w, h, color):
        """Draw a rectangle.

        Args:
            x (int): Starting X position.
            y (int): Starting Y position.
            w (int): Width of rectangle.
            h (int): Height of rectangle.
            color (int): RGB565 color value.
        """
        x2 = x + w - 1
        y2 = y + h - 1
        self.draw_hline(x, y, w, color)
        self.draw_hline(x, y2, w, color)
```

```
        self.draw_vline(x, y, h, color)
        self.draw_vline(x2, y, h, color)

    def draw_sprite(self, buf, x, y, w, h):
        """Draw a sprite (optimized for horizontal drawing).

        Args:
            buf (bytearray): Buffer to draw.
            x (int): Starting X position.
            y (int): Starting Y position.
            w (int): Width of drawing.
            h (int): Height of drawing.
        """
        x2 = x + w - 1
        y2 = y + h - 1
        if self.is_off_grid(x, y, x2, y2):
            return
        self.block(x, y, x2, y2, buf)

    def draw_text(self, x, y, text, font, color,  background=0,
                  landscape=False, spacing=1):
        """Draw text.

        Args:
            x (int): Starting X position.
            y (int): Starting Y position.
            text (string): Text to draw.
            font (XglcdFont object): Font.
            color (int): RGB565 color value.
            background (int): RGB565 background color (default: black).
            landscape (bool): Orientation (default: False = portrait)
            spacing (int): Pixels between letters (default: 1)
        """
        for letter in text:
            # Get letter array and letter dimensions
            w, h = self.draw_letter(x, y, letter, font, color, background,
                                    landscape)
            # Stop on error
            if w == 0 or h == 0:
                print('Invalid width {0} or height {1}'.format(w, h))
                return

            if landscape:
                # Fill in spacing
                if spacing:
                    self.fill_hrect(x, y - w - spacing, h, spacing, background)
                # Position y for next letter
                y -= (w + spacing)
```

```
        else:
            # Fill in spacing
            if spacing:
                self.fill_hrect(x + w, y, spacing, h, background)
            # Position x for next letter
            x += (w + spacing)

            # # Fill in spacing
            # if spacing:
            #     self.fill_vrect(x + w, y, spacing, h, background)
            # # Position x for next letter
            # x += w + spacing

    def draw_text8x8(self, x, y, text, color,  background=0,
                     rotate=0):
        """Draw text using built-in MicroPython 8x8 bit font.

        Args:
            x (int): Starting X position.
            y (int): Starting Y position.
            text (string): Text to draw.
            color (int): RGB565 color value.
            background (int): RGB565 background color (default: black).
            rotate(int): 0, 90, 180, 270
        """
        w = len(text) * 8
        h = 8
        # Confirm coordinates in boundary
        if self.is_off_grid(x, y, x + 7, y + 7):
            return
        # Rearrange color
        r = (color & 0xF800) >> 8
        g = (color & 0x07E0) >> 3
        b = (color & 0x1F) << 3
        buf = bytearray(w * 16)
        fbuf = FrameBuffer(buf, w, h, RGB565)
        if background != 0:
            bg_r = (background & 0xF800) >> 8
            bg_g = (background & 0x07E0) >> 3
            bg_b = (background & 0x1F) << 3
            fbuf.fill(color565(bg_b, bg_r, bg_g))
        fbuf.text(text, 0, 0, color565(b, r, g))
        if rotate == 0:
            self.block(x, y, x + w - 1, y + (h - 1), buf)
        elif rotate == 90:
            buf2 = bytearray(w * 16)
            fbuf2 = FrameBuffer(buf2, h, w, RGB565)
            for y1 in range(h):
```

```
                for x1 in range(w):
                    fbuf2.pixel(y1, x1,
                                fbuf.pixel(x1, (h - 1) - y1))
            self.block(x, y, x + (h - 1), y + w - 1, buf2)
        elif rotate == 180:
            buf2 = bytearray(w * 16)
            fbuf2 = FrameBuffer(buf2, w, h, RGB565)
            for y1 in range(h):
                for x1 in range(w):
                    fbuf2.pixel(x1, y1,
                                fbuf.pixel((w - 1) - x1, (h - 1) - y1))
            self.block(x, y, x + w - 1, y + (h - 1), buf2)
        elif rotate == 270:
            buf2 = bytearray(w * 16)
            fbuf2 = FrameBuffer(buf2, h, w, RGB565)
            for y1 in range(h):
                for x1 in range(w):
                    fbuf2.pixel(y1, x1,
                                fbuf.pixel((w - 1) - x1, y1))
            self.block(x, y, x + (h - 1), y + w - 1, buf2)

    def draw_vline(self, x, y, h, color):
        """Draw a vertical line.

        Args:
            x (int): Starting X position.
            y (int): Starting Y position.
            h (int): Height of line.
            color (int): RGB565 color value.
        """
        # Confirm coordinates in boundary
        if self.is_off_grid(x, y, x, y + h - 1):
            return
        line = color.to_bytes(2, 'big') * h
        self.block(x, y, x, y + h - 1, line)

    def fill_circle(self, x0, y0, r, color):
        """Draw a filled circle.

        Args:
            x0 (int): X coordinate of center point.
            y0 (int): Y coordinate of center point.
            r (int): Radius.
            color (int): RGB565 color value.
        """
        f = 1 - r
        dx = 1
        dy = -r - r
```

```
        x = 0
        y = r
        self.draw_vline(x0, y0 - r, 2 * r + 1, color)
        while x < y:
            if f >= 0:
                y -= 1
                dy += 2
                f += dy
            x += 1
            dx += 2
            f += dx
            self.draw_vline(x0 + x, y0 - y, 2 * y + 1, color)
            self.draw_vline(x0 - x, y0 - y, 2 * y + 1, color)
            self.draw_vline(x0 - y, y0 - x, 2 * x + 1, color)
            self.draw_vline(x0 + y, y0 - x, 2 * x + 1, color)

    def fill_ellipse(self, x0, y0, a, b, color):
        """Draw a filled ellipse.

        Args:
            x0, y0 (int): Coordinates of center point.
            a (int): Semi axis horizontal.
            b (int): Semi axis vertical.
            color (int): RGB565 color value.
        Note:
            The center point is the center of the x0,y0 pixel.
            Since pixels are not divisible, the axes are integer rounded
            up to complete on a full pixel.  Therefore the major and
            minor axes are increased by 1.
        """
        a2 = a * a
        b2 = b * b
        twoa2 = a2 + a2
        twob2 = b2 + b2
        x = 0
        y = b
        px = 0
        py = twoa2 * y
        # Plot initial points
        self.draw_line(x0, y0 - y, x0, y0 + y, color)
        # Region 1
        p = round(b2 - (a2 * b) + (0.25 * a2))
        while px < py:
            x += 1
            px += twob2
            if p < 0:
                p += b2 + px
            else:
```

```
            y -= 1
            py -= twoa2
            p += b2 + px - py
        self.draw_line(x0 + x, y0 - y, x0 + x, y0 + y, color)
        self.draw_line(x0 - x, y0 - y, x0 - x, y0 + y, color)
    # Region 2
    p = round(b2 * (x + 0.5) * (x + 0.5) +
              a2 * (y - 1) * (y - 1) - a2 * b2)
    while y > 0:
        y -= 1
        py -= twoa2
        if p > 0:
            p += a2 - py
        else:
            x += 1
            px += twob2
            p += a2 - py + px
        self.draw_line(x0 + x, y0 - y, x0 + x, y0 + y, color)
        self.draw_line(x0 - x, y0 - y, x0 - x, y0 + y, color)

def fill_hrect(self, x, y, w, h, color):
    """Draw a filled rectangle (optimized for horizontal drawing).

    Args:
        x (int): Starting X position.
        y (int): Starting Y position.
        w (int): Width of rectangle.
        h (int): Height of rectangle.
        color (int): RGB565 color value.
    """
    if self.is_off_grid(x, y, x + w - 1, y + h - 1):
        return
    chunk_height = 1024 // w
    chunk_count, remainder = divmod(h, chunk_height)
    chunk_size = chunk_height * w
    chunk_y = y
    if chunk_count:
        buf = color.to_bytes(2, 'big') * chunk_size
        for c in range(0, chunk_count):
            self.block(x, chunk_y,
                       x + w - 1, chunk_y + chunk_height - 1,
                       buf)
            chunk_y += chunk_height

    if remainder:
        buf = color.to_bytes(2, 'big') * remainder * w
        self.block(x, chunk_y,
                   x + w - 1, chunk_y + remainder - 1,
```

```
                    buf)

    def fill_rectangle(self, x, y, w, h, color):
        """Draw a filled rectangle.

        Args:
            x (int): Starting X position.
            y (int): Starting Y position.
            w (int): Width of rectangle.
            h (int): Height of rectangle.
            color (int): RGB565 color value.
        """
        if self.is_off_grid(x, y, x + w - 1, y + h - 1):
            return
        if w > h:
            self.fill_hrect(x, y, w, h, color)
        else:
            self.fill_vrect(x, y, w, h, color)

    def fill_polygon(self, sides, x0, y0, r, color, rotate=0):
        """Draw a filled n-sided regular polygon.

        Args:
            sides (int): Number of polygon sides.
            x0, y0 (int): Coordinates of center point.
            r (int): Radius.
            color (int): RGB565 color value.
            rotate (Optional float): Rotation in degrees relative to origin.
        Note:
            The center point is the center of the x0,y0 pixel.
            Since pixels are not divisible, the radius is integer rounded
            up to complete on a full pixel.  Therefore diameter = 2 x r + 1.
        """
        # Determine side coordinates
        coords = []
        theta = radians(rotate)
        n = sides + 1
        for s in range(n):
            t = 2.0 * pi * s / sides + theta
            coords.append([int(r * cos(t) + x0), int(r * sin(t) + y0)])
        # Starting point
        x1, y1 = coords[0]
        # Minimum Maximum X dict
        xdict = {y1: [x1, x1]}
        # Iterate through coordinates
        for row in coords[1:]:
            x2, y2 = row
            xprev, yprev = x2, y2
```

```
# Calculate perimeter
# Check for horizontal side
if y1 == y2:
    if x1 > x2:
        x1, x2 = x2, x1
    if y1 in xdict:
        xdict[y1] = [min(x1, xdict[y1][0]), max(x2, xdict[y1][1])]
    else:
        xdict[y1] = [x1, x2]
    x1, y1 = xprev, yprev
    continue
# Non horizontal side
# Changes in x, y
dx = x2 - x1
dy = y2 - y1
# Determine how steep the line is
is_steep = abs(dy) > abs(dx)
# Rotate line
if is_steep:
    x1, y1 = y1, x1
    x2, y2 = y2, x2
# Swap start and end points if necessary
if x1 > x2:
    x1, x2 = x2, x1
    y1, y2 = y2, y1
# Recalculate differentials
dx = x2 - x1
dy = y2 - y1
# Calculate error
error = dx >> 1
ystep = 1 if y1 < y2 else -1
y = y1
# Calcualte minimum and maximum x values
for x in range(x1, x2 + 1):
    if is_steep:
        if x in xdict:
            xdict[x] = [min(y, xdict[x][0]), max(y, xdict[x][1])]
        else:
            xdict[x] = [y, y]
    else:
        if y in xdict:
            xdict[y] = [min(x, xdict[y][0]), max(x, xdict[y][1])]
        else:
            xdict[y] = [x, x]
    error -= abs(dy)
    if error < 0:
        y += ystep
        error += dx
```

```
            x1, y1 = xprev, yprev
        # Fill polygon
        for y, x in xdict.items():
            self.draw_hline(x[0], y, x[1] - x[0] + 2, color)

    def fill_vrect(self, x, y, w, h, color):
        """Draw a filled rectangle (optimized for vertical drawing).

        Args:
            x (int): Starting X position.
            y (int): Starting Y position.
            w (int): Width of rectangle.
            h (int): Height of rectangle.
            color (int): RGB565 color value.
        """
        if self.is_off_grid(x, y, x + w - 1, y + h - 1):
            return
        chunk_width = 1024 // h
        chunk_count, remainder = divmod(w, chunk_width)
        chunk_size = chunk_width * h
        chunk_x = x
        if chunk_count:
            buf = color.to_bytes(2, 'big') * chunk_size
            for c in range(0, chunk_count):
                self.block(chunk_x, y,
                           chunk_x + chunk_width - 1, y + h - 1,
                           buf)
                chunk_x += chunk_width

        if remainder:
            buf = color.to_bytes(2, 'big') * remainder * h
            self.block(chunk_x, y,
                       chunk_x + remainder - 1, y + h - 1,
                       buf)

    def is_off_grid(self, xmin, ymin, xmax, ymax):
        """Check if coordinates extend past display boundaries.

        Args:
            xmin (int): Minimum horizontal pixel.
            ymin (int): Minimum vertical pixel.
            xmax (int): Maximum horizontal pixel.
            ymax (int): Maximum vertical pixel.
        Returns:
            boolean: False = Coordinates OK, True = Error.
        """
        if xmin < 0:
            print('x-coordinate: {0} below minimum of 0.'.format(xmin))
```

```
            return True
        if ymin < 0:
            print('y-coordinate: {0} below minimum of 0.'.format(ymin))
            return True
        if xmax >= self.width:
            print('x-coordinate: {0} above maximum of {1}.'.format(
                xmax, self.width - 1))
            return True
        if ymax >= self.height:
            print('y-coordinate: {0} above maximum of {1}.'.format(
                ymax, self.height - 1))
            return True
        return False

    def load_sprite(self, path, w, h):
        """Load sprite image.

        Args:
            path (string): Image file path.
            w (int): Width of image.
            h (int): Height of image.
        Notes:
            w x h cannot exceed 2048
        """
        buf_size = w * h * 2
        with open(path, "rb") as f:
            return f.read(buf_size)

    def reset_cpy(self):
        """Perform reset: Low=initialization, High=normal operation.

        Notes: CircuitPython implemntation
        """
        self.rst.value = False
        sleep(.05)
        self.rst.value = True
        sleep(.05)

    def reset_mpy(self):
        """Perform reset: Low=initialization, High=normal operation.

        Notes: MicroPython implemntation
        """
        self.rst(0)
        sleep(.05)
        self.rst(1)
        sleep(.05)
```

```
    def scroll(self, y):
        """Scroll display vertically.

        Args:
            y (int): Number of pixels to scroll display.
        """
        self.write_cmd(self.VSCRSADD, y >> 8, y & 0xFF)

    def set_scroll(self, top, bottom):
        """Set the height of the top and bottom scroll margins.

        Args:
            top (int): Height of top scroll margin
            bottom (int): Height of bottom scroll margin
        """
        if top + bottom <= self.height:
            middle = self.height - (top + bottom)
            print(top, middle, bottom)
            self.write_cmd(self.VSCRDEF,
                           top >> 8,
                           top & 0xFF,
                           middle >> 8,
                           middle & 0xFF,
                           bottom >> 8,
                           bottom & 0xFF)

    def sleep(self, enable=True):
        """Enters or exits sleep mode.

        Args:
            enable (bool): True (default)=Enter sleep mode, False=Exit sleep
        """
        if enable:
            self.write_cmd(self.SLPIN)
        else:
            self.write_cmd(self.SLPOUT)


    def write_cmd_mpy(self, command, *args):
        """Write command to OLED (MicroPython).

        Args:
            command (byte): ILI9341 command code.
            *args (optional bytes): Data to transmit.
        """
        self.dc(0)
        self.cs(0)
        self.spi.write(bytearray([command]))
        self.cs(1)
```

```
        # Handle any passed data
        if len(args) > 0:
            self.write_data(bytearray(args))

    def write_cmd_cpy(self, command, *args):
        """Write command to OLED (CircuitPython).

        Args:
            command (byte): ILI9341 command code.
            *args (optional bytes): Data to transmit.
        """
        self.dc.value = False
        self.cs.value = False
        # Confirm SPI locked before writing
        while not self.spi.try_lock():
            pass
        self.spi.write(bytearray([command]))
        self.spi.unlock()
        self.cs.value = True
        # Handle any passed data
        if len(args) > 0:
            self.write_data(bytearray(args))

    def write_data_mpy(self, data):
        """Write data to OLED (MicroPython).

        Args:
            data (bytes): Data to transmit.
        """
        self.dc(1)
        self.cs(0)
        self.spi.write(data)
        self.cs(1)

    def write_data_cpy(self, data):
        """Write data to OLED (CircuitPython).

        Args:
            data (bytes): Data to transmit.
        """
        self.dc.value = True
        self.cs.value = False
        # Confirm SPI locked before writing
        while not self.spi.try_lock():
            pass
        self.spi.write(data)
        self.spi.unlock()
        self.cs.value = True
```